0 1 2 3 4 5 6 7 8 9 10 11

2 8 11 A E I N Q O E S S T U Y

2 3 7 A E E I Q O N S S T U Y

2 2 A E E I Q O N S S T U Y

4 6 7 A E E I N O Q S S T U Y

4 5 5 A E E I O N Q S S T U Y

4 4 A E E I O N Q S S T U Y

6 6 A E E I O N Q S S T U Y

9 9 11 A E E I O N Q S S T U Y

10 10 11 A E E I O N Q S S T U Y

11 11 A E E I O N Q S S T U Y

2.3.3 : Khi chạy QS cho mảng độ dài N, phần tử max sẽ bị đổi chỗ nhiều nhất chỉ 1 lần

2.3.4 . Bỏ qua trộn ngẫu nhiên bđ, mảng s/x tăng dần sẽ là mảng mà QS dùng nhiều p/ss nhất

2.3.5 . Khi bt mảng chỉ có 2 gtri.

```
public static void sortTwoVal(Comparable[] a) {
    Comparable min = a[0]
    for (int i = 1; i < a.length ; i++) :
        int cmp = a[0].compareTo(a[i])
        if (cmp > 0) min = a[i] ; break;
        else if (cmp < 0) break;
    int i = 0 , j = a.length - 1
    while (i < j) :
        while (i < j and a[i].compareTo(min) == 0) i++;
        while (i < j and a[j].compareTo(min) > 0) ; j--
        if (i < j)
            Comparable tmp = a[i]
            a[i] = a[j]
            a[j] = tmp ; i++ ; j--
```

Vg lặp để tìm min tối đa O(N), vg lặp phân hoạch tốn
tối N p ss. → Tg t/g chạy là O(N).
Bộ nhớ: TT in-place → memory O(1)

2.3.7.
2.3.8. Nếu tất cả cái phần tử trg 1 mảg N = nhau
Với vc chọn pivot là a[lo]
~~Sau vg lặp phân hoạch đầu tiên, sẽ chạy đúng 2N phép ss~~
~~Tg tự vg lặp t2 sẽ mất 2N-2 p ss~~
Sau N lần lặp
tt sẽ k tra đk less(a[++i], a[lo]) và less(a[lo], a[
luôn trả về false và sẽ đổi chỗ thực ngay sau khi ++i và --j đi 1 lần.

Ta xét:
Tầng 1: Đỉnh: xử lý mảg N ptử, phoạch tốn N
Tầng 2: Chia làm 2 mảg N/2 → phân hoạch tốn $2 \times \frac{N}{2} = N$
Tg tự . . .
Tầng k (Đáy) xử lí N mảg con kt 1, e tốn c phí nào
⇒ N → N/2 → N/4 . . . 1
→ số lần mảg có thể chia đôi là $\log_2 N$.
→ Tg c phí = cp 1 tầng × số tầng
= $N\log_2 N$

TH worst Case: Nếu mảg đã đc s/x sẵn hay là s/x ngc thì sẽ là $\sim \frac{1}{2}N^2$.

NN: tầng 1: Xử lí mảg N ptử → N cp.
e mảg phụ bị lệch về hẳn 1 bên
Tầng 2: Xử lí mảg N-1 ptử → N-1 cp.
t² đẩy về 1 bên,
Tầng N (đáy) c còn 1 ptử → 1 p ss
Khi này số lần cắt mảg buộc là N.

→ $T_g = N + (N-1) + (N-2) + \dots + 2 + 1$

$= \frac{1}{2}N^2 + \frac{1}{2}N \sim \frac{1}{2}N^2$

2.3.9. Với mảng có 2 hoặc 3 gtri pb

Với QS basic: Vẫn tốn $\sim N \log N$ phép s^2

Với QS-3W:

- Với 2 gtri pb: lần phân hoạch đầu, chốt sẽ là 1 trg 2, Sau, đó toàn bộ pt = pivot sẽ nằm ở nhóm = v. Và nhóm > v sẽ ở bên phải hoặc < v sẽ ở bên t². Khi ấy, đệ quy chạy vào 1 trg 2 mảng con đó, nó sẽ quét và đưa tất cả vào nhóm = v mg. Rồi kt.
- Với 3 gt pb, cx gần tg tự, pivot có thể rơi vào 1 trg 3 TH, min, medium, high. Dù rơi vào nhóm nào thì tt cx chỉ cần gọi đệ quy tối đa 3 lần.

2.3.11. Q/s dừng QS phân hoạch mà bỏ qua các gtri pt = pivot thay vì dừng

→ Sửa while ($a[i] <= a[lo]$)

và while ($a[j] >= a[lo]$)

Với mảng có tất cả các pt = nhau, chg sẽ tốn đúng N bộ đệ quy với mảng N, N-1, N-2...

→ Cphí là $\sim \frac{1}{2}N^2$.

2.3. Độ sâu gọi đệ quy chính là số lần chia mảng, pt vào cách s/x mảng bđ:

Worst-case: $\sim N$: Khi phần tử pivot luôn rơi vào max or min, tức mảng khi bị cắt lệch

Best-case: $\sim \log_2 N$: Khi chia đôi mảng khi pivot luôn nằm giữa hoàn hảo

Average-case: $O(\log N)$.

Nhờ suffle, mảng sẽ đc chia tg đối đều. Đảm bảo độ sâu đệ quy ở mức logarit với P cao.

```
        QS( compa
    if (      re
    int lt        gt = hi
    parable        [lo]
    i = lo
    (i <= gt
        mp = a[i].compareTo(v).
        if (cmp<0) : exch(a, lt++, i++).
        eif (cmp>0) : exch(a, gt--, i).
        else : i++.
    }
    sort(a, lo, lt-1)  //(a[lt] đến a[gt] đã
    sort(a, gt+1, hi)  // nằm đúng vị trí).
}
```

→ Tg chạy O(NlogN) → O(N).

Quick Sort.

v 2.3.1:

→ E A S Y QUESTION.

sc left, sc right: E A S Y QUESTION

final exch

result: A E S Y QUESTION.

2.3.2.

lo	j	hi	0	1	2	3	4	5	6	7	8	9	1
			E	A	S	Y	Q	U	E	S	T	I	O
0	1	~~11~~	A	E	S	Y	Q	U	E	S	T	I	O
0.		~~0~~	A	E	S	Y	Q	U	E	S	T	I	O
						↑ i							